Practical – 2

Study of pre-processing methods:-
Write a program to find following statistics from a given dataset. Mean, mode, median, variance, Remarks standard deviation, quartiles, interquartile range.

1. Program to find mean, mode, median, variance, and standard deviation of given dataset:

Sample Dataset

```
data = [10, 20, 20, 30, 40, 570]
```

a. **Mean**:- The mean is the average value of a dataset. It is calculated by adding all observations and dividing by the total number of observations. It represents the central tendency of the data.

[3]
✓ 0s

```
#Mean
import numpy as np
mean = np.mean(data)
print("Mean:", mean)
```

```
Mean: 115.0
```

b. **Median**:- The median is the middle value of a dataset when the data is arranged in ascending order. If the number of observations is even, the median is the average of the two middle values.

[4]
✓ 0s

```
#Median
median = np.median(data)
print("Median:", median)
```

```
Median: 25.0
```

c. **Mode**:- The mode is the value that occurs most frequently in a dataset. A dataset can have one mode, more than one mode, or no mode at all.

```
[5]
✓ 0s
#Mode
from statistics import mode
mode_value = mode(data)
print("Mode:", mode_value)

Mode: 20
```

d. **Variance**:- The variance measures how much the data values are spread out from the mean. A higher variance indicates greater dispersion in the data.

```
[6]
✓ 0s
#Variance
variance = np.var(data)
print("Variance:", variance)

Variance: 41491.666666666664
```

e. **Standard Deviation**:- The standard deviation is the square root of the variance. It shows how much the data values typically deviate from the mean and is expressed in the same units as the data.

```
[7]
✓ 0s
#Standard Deviation
std_dev = np.std(data)
print("Standard Deviation:", std_dev)

Standard Deviation: 203.6950334855189
```

f. **Quartiles**:- Quartiles divide a sorted dataset into four equal parts:

- Q1 (First Quartile): 25% of data lies below this value
- Q2 (Second Quartile): 50% of data lies below this value (Median)
- Q3 (Third Quartile): 75% of data lies below this value

```
[8]
✓ 0s
#Quartile (25, 50, 75)
Q1 = np.percentile(data, 25)
Q2 = np.percentile(data, 50)
Q3 = np.percentile(data, 75)

print("Q1:", Q1)
print("Q2 (Median):", Q2)
print("Q3:", Q3)

Q1: 20.0
Q2 (Median): 25.0
Q3: 37.5
```

g. **Interquartile Range(IQR)**:- The interquartile range is the difference between the third quartile (Q3) and the first quartile (Q1). It measures the spread of the middle 50% of the data and helps in detecting outliers.

```
[9]
✓ 0s
#Interquartile
IQR = Q3 - Q1
print("Interquartile Range (IQR):", IQR)

Interquartile Range (IQR): 17.5
```

2. Handling Missing Values With Mean, Median, and Mode:

Handling missing values is an important step in data preprocessing to ensure data quality and reliable model performance.

- **Mean Imputation:**
  Missing values are replaced with the **mean** of the column. This method is suitable for **numerical data** with no extreme outliers, as the mean represents the average value.
- **Median Imputation:**
  Missing values are replaced with the **median** of the column. This method is preferred when the data contains **outliers or skewed distributions**, since the median is less affected by extreme values.
- **Mode Imputation:**
  Missing values are replaced with the **mode** (most frequent value) of the column. This method is commonly used for **categorical data**, where numerical averages are not meaningful.

Dataset Preparation

```
[10]
✓ 1s
import pandas as pd
import numpy as np

data = {
    "Monthly_Usage_GB": [5, 12, np.nan, 20, 50, 8, np.nan, 100],
    "Contract_Type": ["Prepaid", "Postpaid", "Postpaid", "Prepaid", "Postpaid", np.nan, "Prepaid", "Postpaid"],
    "Payment_Method": ["UPI", "Card", "Cash", "UPI", "Card", "Cash", np.nan, "UPI"],
    "Customer_Tenure_Months": [2, 12, 6, 24, np.nan, 4, 1, 60],
    "Monthly_Bill": [299, 499, 399, 599, 999, np.nan, 249, 1999],
    "Churn": ["No", "No", "Yes", "No", "Yes", "No", "No", "Yes"]
}

df = pd.DataFrame(data)
df.head()
```

	Monthly_Usage_GB	Contract_Type	Payment_Method	Customer_Tenure_Months	Monthly_Bill	Churn
0	5.0	Prepaid	UPI	2.0	299.0	No
1	12.0	Postpaid	Card	12.0	499.0	No
2	NaN	Postpaid	Cash	6.0	399.0	Yes

Handling Missing Values

```python
#missing values
print(df.isnull().sum())
#mean
df["Monthly_Usage_GB"] = df["Monthly_Usage_GB"].fillna(
    df["Monthly_Usage_GB"].mean()
)
#median
df["Customer_Tenure_Months"] = df["Customer_Tenure_Months"].fillna(
    df["Customer_Tenure_Months"].median()
)
df["Monthly_Bill"] = df["Monthly_Bill"].fillna(
    df["Monthly_Bill"].median()
)
#mode
df["Contract_Type"] = df["Contract_Type"].fillna(
    df["Contract_Type"].mode()[0]
)
df["Payment_Method"] = df["Payment_Method"].fillna(
    df["Payment_Method"].mode()[0]
)
print("Dataset after handling the missing values:", df.isnull().sum())
```

```
Monthly_Usage_GB          2
Contract_Type             1
Payment_Method            1
Customer_Tenure_Months    1
Monthly_Bill              1
Churn                     0
dtype: int64
Dataset after handling the missing values: Monthly_Usage_GB          0
Contract_Type             0
Payment_Method            0
Customer_Tenure_Months    0
Monthly_Bill              0
Churn                     0
dtype: int64
```

3. One-Hot Encoding:

One-hot encoding is a technique used to convert **categorical variables into numerical format** by creating separate binary columns for each unique category. Each column represents one category and contains values of **1 (present)** or **0 (absent)**.

This method prevents the model from assuming any **ordinal relationship** between categories, making it especially suitable for **nominal categorical data** such as colors, cities, or types. However, one-hot encoding can increase the number of features when a column has many unique values.

One-Hot Encoding

```python
df = pd.get_dummies(df, columns=["Payment_Method"], drop_first=True)

df
```

	Monthly_Usage_GB	Contract_Type	Customer_Tenure_Months	Monthly_Bill	Churn	Payment_Method_Cash	Payment_Method_UPI
0	5.0	1	2.0	299.0	0	False	True
1	12.0	0	12.0	499.0	0	False	False
2	32.5	0	6.0	399.0	1	True	False
3	20.0	1	24.0	599.0	0	False	True
4	50.0	0	6.0	999.0	1	False	False
5	8.0	0	4.0	499.0	0	True	False
6	32.5	1	1.0	249.0	0	False	True
7	100.0	0	60.0	1999.0	1	False	True

4. Label Encoding:

Label encoding is a data preprocessing technique used to convert **categorical values into numerical form**. Each unique category in a column is assigned a unique integer value. This allows machine learning algorithms to work with categorical data.

Label encoding is mainly used for **ordinal categorical data**, where the categories have a meaningful order (e.g., *Low, Medium, High*). For nominal data with no inherent order, label encoding may introduce unintended relationships.

Label Encoding

[12]

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["Contract_Type"] = le.fit_transform(df["Contract_Type"])
df["Churn"] = le.fit_transform(df["Churn"])

df.head()
```

	Monthly_Usage_GB	Contract_Type	Payment_Method	Customer_Tenure_Months	Monthly_Bill	Churn
0	5.0	1	UPI	2.0	299.0	0
1	12.0	0	Card	12.0	499.0	0
2	NaN	0	Cash	6.0	399.0	1
3	20.0	1	UPI	24.0	599.0	0
4	50.0	0	Card	NaN	999.0	1

5. Interquartile Range(IQR):

The **Interquartile Range (IQR)** is a statistical measure used to describe the **spread of the middle 50% of a dataset**. It is calculated as the difference between the **third quartile (Q3)** and the **first quartile (Q1)**.

IQR

[13]

```
def apply_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    return df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]
```

[ ]

```
num_cols = [
    "Monthly_Usage_GB",
    "Customer_Tenure_Months",
    "Monthly_Bill"
]

for col in num_cols:
    df = apply_iqr(df, col)
df
```

	Monthly_Usage_GB	Contract_Type	Customer_Tenure_Months	Monthly_Bill	Churn	Payment_Method_Cash	Payment_Method_UPI
0	5.0	1	2.0	299.0	0	False	True
1	12.0	0	12.0	499.0	0	False	False
2	32.5	0	6.0	399.0	1	True	False
5	8.0	0	4.0	499.0	0	True	False
6	32.5	1	1.0	249.0	0	False	True

6. Normalization:

Normalization is a data preprocessing technique used to **scale numerical features to a common range**, usually between **0 and 1**. It ensures that all features contribute equally to the model and prevents features with larger values from dominating those with smaller values.

Normalization

```
X = df.drop("Churn", axis=1)
y = df["Churn"]
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

num_cols = ["Monthly_Usage_GB", "Customer_Tenure_Months", "Monthly_Bill"]

df[num_cols] = scaler.fit_transform(df[num_cols])

df
```

	Monthly_Usage_GB	Contract_Type	Customer_Tenure_Months	Monthly_Bill	Churn	Payment_Method_Cash	Payment_Method_UPI
0	0.000000	1	0.090909	0.2	0	False	True
1	0.254545	0	1.000000	1.0	0	False	False
2	1.000000	0	0.454545	0.6	1	True	False
5	0.109091	0	0.272727	1.0	0	True	False

7. Regularization (Lasso):

Lasso regularization is a technique used to **prevent overfitting** by adding a penalty equal to the **absolute values of model coefficients** to the loss function. This penalty discourages large coefficients and can shrink some coefficients **exactly to zero**.

Because of this property, Lasso not only reduces model complexity but also performs **feature selection** by eliminating less important features. Lasso regularization is widely used in **linear and logistic regression models**, especially when dealing with high-dimensional data.

Lasso Regularization (L1 Regularization)

```
from sklearn.linear_model import Lasso
import pandas as pd

# Features & target (remove target-like columns from X)
X = df.drop(["Monthly_Bill", "Churn"], axis=1)
y = df["Monthly_Bill"]

# Lasso Regression (L1 Regularization)
lasso = Lasso(alpha=0.1)
lasso.fit(X, y)

# Coefficients
lasso_coef = pd.DataFrame({
    "Feature": X.columns,
    "Lasso_Coefficient": lasso.coef_
})

lasso_coef
```

	Feature	Lasso_Coefficient
0	Monthly_Usage_GB	-0.000000e+00
1	Contract_Type	-3.500000e-01
2	Customer_Tenure_Months	0.000000e+00
3	Payment_Method_Cash	0.000000e+00
4	Payment_Method_UPI	-2.775558e-16